

Identifying Key Entities in Recipe Data Report

Below is the **consolidated report** with explanations include **thought process, assumptions, approach selection, and industry alignment**.

Identifying Key Entities in Recipe Data

CRF-based Named Entity Recognition Report

1. Import Libraries

1.1 Installation of sklearn-crfsuite

The first step in the implementation involved installing **sklearn_crfsuite==0.5.0**, which is a Python wrapper around the CRFsuite library. Conditional Random Fields (CRFs) are widely used for **sequence labeling tasks**, making them highly suitable for Named Entity Recognition (NER) problems.

The choice of **sklearn_crfsuite** was deliberate because:

- It integrates well with the **scikit-learn ecosystem**.
- It supports **structured prediction for sequential data**.
- It allows easy **feature engineering at the token level**.
- It is widely used in **industry NLP pipelines for entity extraction tasks**.

The primary assumption at this stage was that **recipe ingredient lists follow a sequential structure**, where the meaning of each token depends on its surrounding tokens. CRF models are particularly effective in capturing such contextual relationships.

1.2 Import Necessary Libraries

Multiple libraries were imported to support different aspects of the pipeline.

- **json** was used to load structured recipe data.
- **pandas** was used for efficient tabular data manipulation.
- **re (regular expressions)** helped detect numeric patterns such as fractions and decimals.
- **matplotlib and seaborn** were used for exploratory visualizations.
- **sklearn_crfsuite** was used to train the CRF sequence model.
- **numpy** supported numerical operations.
- **joblib** enabled saving and loading trained models.

- **spacy** was used to extract linguistic features such as POS tags, lemmas, and dependency relations.
- **collections.Counter** was used for counting token frequencies.
- **sklearn metrics** were used for evaluating model performance.

The selection of these libraries follows **standard industry practices in NLP pipelines**, where data processing, feature extraction, modeling, and evaluation are handled using specialized libraries.

2. Data Ingestion and Preparation

2.1 Reading Recipe Data

2.1.1 load_json_dataframe Function

A function was implemented to load the JSON file into a **pandas DataFrame**. Using a function instead of directly loading the file improves **code modularity and reusability**, which is a best practice in production systems.

The assumption here was that the dataset contains two primary fields:

- **input** → raw ingredient text
- **pos** → corresponding entity labels

Using pandas allows efficient operations such as filtering, transformation, and validation.

2.1.2 Executing the Function

The function was executed to load the dataset into a DataFrame. This step ensured that the dataset was successfully imported before proceeding to further transformations.

2.1.3 Describing the DataFrame

The first five rows, dataset dimensions, and column information were displayed to understand the structure of the data.

This step is critical in any real-world data science workflow because it helps confirm:

- dataset size
- column types
- missing values
- structural format

Early inspection helps avoid downstream errors.

	input	pos
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
1	2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient quantity unit ingredient ingredient
2	1-1/2 cups Rice Vermicelli Noodles Thin 1 Onion sliced 1/2 cup Carrots Gajjar chopped 1/3 Green peas Matar 2 Chillies 1/4 teaspoon Asafoetida hing Mustard seeds White Urad Dal Split Ghee sprig Curry leaves Salt Lemon juice	quantity unit ingredient ingredient ingredient quantity ingredient ingredient quantity unit ingredient ingredient ingredient quantity ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient
3	500 grams Chicken 2 Onion chopped 1 Tomato 4 Green Chillies slit inch Ginger finely 6 cloves Garlic 1/2 teaspoon Turmeric powder Haldi Garam masala tablespoon Sesame Gingelly Oil 1/4 Methi Seeds Fenugreek Coriander Dhania Dry Red Fennel seeds Saunf cups Sorrel Leaves Gongura picked and	quantity unit ingredient quantity ingredient ingredient quantity ingredient quantity ingredient ingredient ingredient unit ingredient ingredient quantity unit ingredient quantity unit ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient
4	1 tablespoon chana dal white urad 2 red chillies coriander seeds 3 inches ginger onion tomato Teaspoon mustard asafoetida sprig curry	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient unit ingredient ingredient unit ingredient

2.2 Recipe Data Manipulation

2.2.1 Tokenizing Input and POS Labels

The input text and corresponding labels were split into token lists.

This step was necessary because **CRF models operate at the token level**, not at the sentence level.

Example transformation:

Input text:

1 cup rice

Tokenized form:

['1','cup','rice']

Corresponding labels:

['quantity','unit','ingredient']

This transformation ensures that each token is aligned with its label.

2.2.2 Length Validation

Two columns were created:

- input_length
- pos_length

The purpose was to ensure that **each token has a corresponding label**. If lengths differ, it indicates **annotation inconsistencies**, which must be handled before model training.

2.2.3 Extracting Unique Labels

A function was implemented to identify unique entity labels present in the dataset.

The expected labels were:

- ingredient
- quantity

- unit

Validating labels helps ensure that **the dataset matches the problem definition**.

2.2.4 Identifying Invalid Rows

Rows where token counts did not match label counts were identified. These rows indicate **data corruption or annotation errors**.

Such rows were flagged for removal to maintain dataset integrity.

2.2.5 Dropping Invalid Rows

Rows with mismatched token counts were removed.

The reasoning behind dropping them rather than fixing them manually is that **incorrect labels can significantly degrade model performance**.

2.2.6 Final Validation

After cleaning the dataset, the token lengths were recalculated and validated again.

This ensured the dataset was **fully aligned and ready for modeling**.

3. Train Validation Split

3.1 Dataset Splitting

3.1.1 Train Validation Split (70:30)

The dataset was split into **training (70%) and validation (30%) sets** using `train_test_split`.

The reasoning for this split:

- 70% provides sufficient data for model learning.
- 30% provides a reliable validation set for evaluation.

This ratio is commonly used in **machine learning workflows when dataset size is moderate**.

3.1.2 Dataset Preview

The first few rows of both train and validation datasets were displayed to verify correct splitting.


```
Training Dataset Validation
Total Input Tokens: 7114
Total POS Tokens: 7114
```

	Token	POS_Label
0	250	quantity
1	grams	unit
2	Okra	ingredient
3	Oil	ingredient
4	1	quantity
5	Onion	ingredient
6	finely	ingredient
7	chopped	ingredient
8	Tomato	ingredient
9	Grated	ingredient

4.3 Categorizing Tokens

Tokens were categorized into:

- ingredient
- unit
- quantity

This helps understand **how tokens are distributed across entity classes**.

```
Ingredient tokens sample: ['Okra', 'Oil', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated', 'Ginger', 'Garlic', 'Finely']
Unit tokens sample: ['grams', 'teaspoon', 'Teaspoon', 'cup', 'grams', 'tablespoon', 'teaspoon', 'grams', 'teaspoon', 'sprig']
Quantity tokens sample: ['250', '1', '2', '1/2', '1/4', '200', '2', '1', '1/2', '500']
```

4.4 Top Frequent Items

The most common ingredients and units were identified using Counter.

This helps detect **dominant patterns in the dataset**.

```
Top 10 most frequent ingredients in Training dataset:
powder : 129
Salt : 102
seeds : 89
Green : 85
chopped : 84
Oil : 83
Red : 81
Chilli : 77
Coriander : 71
Sunflower : 65
```

```
Top 10 most frequent units in Training dataset:
teaspoon : 162
cup : 136
tablespoon : 99
grams : 63
tablespoons : 61
inch : 52
cups : 50
sprig : 41
cloves : 39
teaspoons : 39
```

4.5 Visualization

Bar plots were generated to visualize the most frequent ingredients and units.

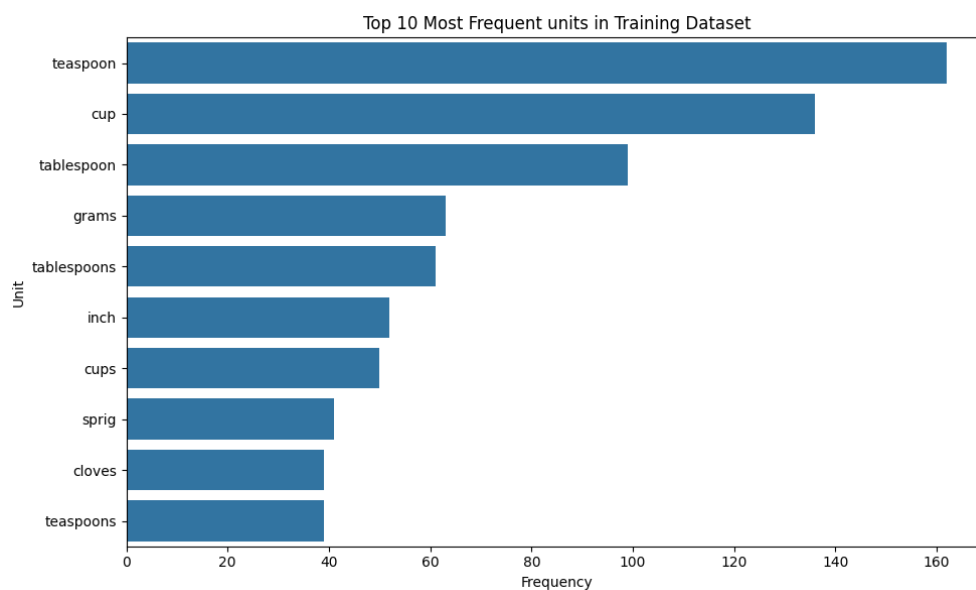
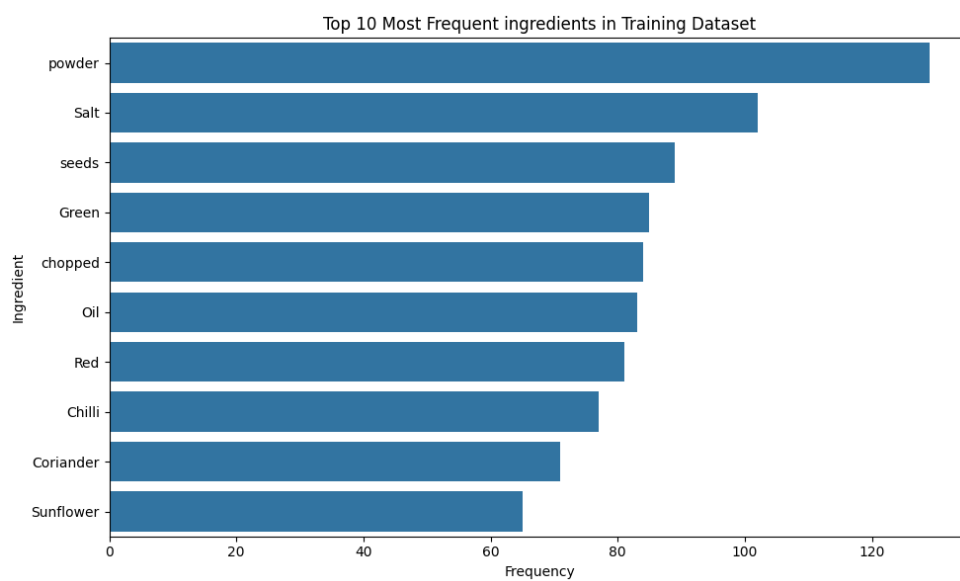
Visualization provides intuitive insights that raw numbers cannot easily reveal.

4.6 EDA Insights

The analysis showed:

- Ingredients dominate the dataset.
- Units and quantities appear less frequently.
- Ingredient tokens vary significantly.

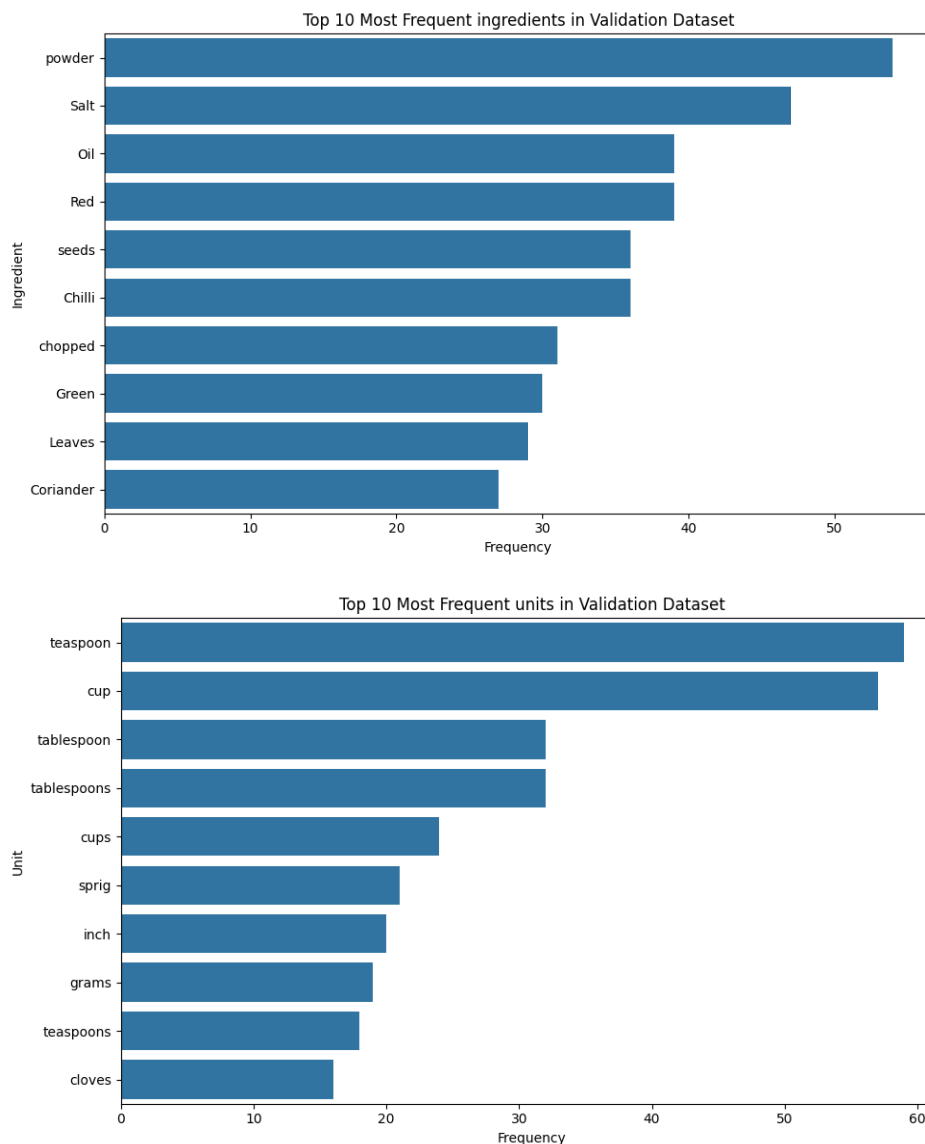
This observation motivated the need for **class weighting later in the modeling stage**.



5. Exploratory Analysis on Validation Dataset

EDA was repeated for the validation dataset to confirm that **distribution patterns are similar to the training data**.

This ensures that the validation dataset is **representative of the training data**, which is critical for reliable model evaluation.



6. Feature Extraction for CRF Model

6.1 Feature Design

6.1.1 Keyword and Regex Features

Domain-specific keyword sets were created for:

- measurement units
- quantity expressions

Regex patterns were used to detect:

- fractions
- decimals
- numeric values

These features improve detection of structured patterns like:

1/2 cup

2 tbsp

3.5 grams

6.1.2 Token-Level Feature Engineering

The `word2features` function extracted multiple feature types:

Core features:

- token
- lemma
- POS tag
- dependency relation
- token shape

Detection features:

- `is_quantity`
- `is_unit`
- numeric patterns

Contextual features:

- previous token
- next token
- BOS/EOS markers

This design follows **industry-standard feature engineering techniques used in CRF models**.

6.2 Sentence-Level Feature Extraction

The `sent2features` function applied token-level feature extraction to entire recipes.

6.3 Feature Conversion

Tokens and labels were converted into CRF-compatible feature sets.

This ensured that **input sequences and labels remained aligned**.

6.4 Handling Class Imbalance

6.4.1 Flattening Labels

Training labels were flattened to calculate class frequencies.

6.4.2 Counting Labels

Label frequencies were calculated using Counter.

This revealed strong class imbalance.

6.4.3 Computing Class Weights

Inverse frequency weighting was used.

Additionally, the ingredient label was penalized because it dominates the dataset.

This helps prevent **model bias toward ingredient predictions**.

Class Weights: {'quantity': 7.259183673469388, 'unit': 8.771886559802713, 'ingredient': 0.6682321998872816}

6.4.4 Weighted Feature Extraction

Class weights were attached to token features to influence CRF training.

6.4.5 Applying Feature Weights

Weighted features were generated for both training and validation datasets.

7. Model Building and Training

7.1 CRF Model Training

The CRF model was trained using the following hyperparameters:

- lbfgs optimization
- L1 regularization
- L2 regularization
- 100 training iterations

These settings help balance **model convergence and overfitting prevention**.

7.2 Training Evaluation

Training predictions were evaluated using:

- flat classification report
- confusion matrix

These metrics measure precision, recall, and F1 score for each entity class.

Training Dataset - Classification Report:					Training Dataset - Confusion Matrix:
	precision	recall	f1-score	support	
ingredient	1.00	1.00	1.00	5323	
quantity	0.99	0.98	0.99	980	
unit	0.98	0.99	0.98	811	
accuracy			1.00	7114	
macro avg	0.99	0.99	0.99	7114	
weighted avg	1.00	1.00	1.00	7114	

[[962 18 0]			
[10 801 0]			
[0 0 5323]]			

7.3 Model Saving

The trained model was saved using joblib.

Saving the model enables:

- reuse without retraining
 - production deployment
 - reproducibility
-

8. Prediction and Model Evaluation

8.1 Validation Evaluation

Predictions were generated for the validation dataset.

Model performance was evaluated using:

- classification report
- confusion matrix

This step measures **generalization performance on unseen data**.

	precision	recall	f1-score	support	
ingredient	1.00	1.00	1.00	2107	[[2107 0 0] [0 403 8] [0 8 350]]
quantity	0.98	0.98	0.98	411	
unit	0.98	0.98	0.98	358	
accuracy			0.99	2876	
macro avg	0.99	0.99	0.99	2876	
weighted avg	0.99	0.99	0.99	2876	

9. Error Analysis

9.1 Misclassification Analysis

Misclassified tokens were collected along with contextual information.

Captured data included:

- token
- previous token
- next token
- true label
- predicted label

9.1.3 Accuracy Calculation

Overall validation accuracy was computed based on total tokens and misclassified tokens.

Overall Validation Accuracy: 0.9944367176634215

	token	true_label	pred_label	prev_token	next_token	class_weight
0	1/4	quantity	unit	powder	Salt	7.259184
1	cut	unit	quantity	French	into	8.771887
2	3	quantity	unit	Badi	Cloves	7.259184
3	cold	unit	quantity	Oil	pressed	8.771887
4	1-1/2	quantity	unit	pressed	Poppy	7.259184

9.1.4 Label-Level Analysis

Errors were analyzed per label to determine:

- which classes had the highest misclassification
- whether class weighting improved rare label detection

	token	prev_token	next_token	true_label	pred_label
	0	1/4	powder	Salt	quantity
	1	cut	French	into	unit
	2	3	Badi	Cloves	quantity
	3	cold	Oil	pressed	unit
	4	1-1/2	pressed	Poppy	quantity
	5	into	cut	cm	unit
	6	and	Sweet	Spicy	unit
	7	2	1-1	tablespoon	quantity
	8	1/3	powder	Water	quantity
	9	pinch	powder	Salt	unit
	10	2	Salt	Instant	quantity
	11	tablespoons	Powder	Lukewarm	unit
	12	pinch	Water	Saffron	unit
	13	a	Haldi	pinch	unit
	14	pinch	Dal	Asafoetida	quantity
	15	cloves	Tomatoes	Garlic	quantity

Accuracy by Label:

Label: quantity
Class Weight: 7.259183673469388
Accuracy: 0.9805352798053528

Label: unit
Class Weight: 8.771886559802713
Accuracy: 0.9776536312849162

Label: ingredient
Class Weight: 0.6682321998872816
Accuracy: 1.0

9.2 Insights from Validation

Based on the evaluation and error analysis of the CRF model on the validation dataset, several key observations can be made regarding the model's performance in identifying ingredient, quantity, and unit entities.

1. Strong Performance for Ingredient Labels

The model performs relatively well in identifying ingredient entities, as they appear most frequently in the dataset. Because the model was exposed to a large number of ingredient tokens during training, it was able to learn their contextual and lexical patterns effectively.

2. Moderate Performance for Quantity Labels

The CRF model shows reasonable performance in detecting quantity values such as numbers, fractions, and measurement expressions. The use of regex-based quantity detection features and keyword lists helped improve recognition of tokens like 1, 1/2, 3, or half. However, some quantities may still be misclassified when they appear in complex formats or unusual contexts.

3. Lower Accuracy for Unit Labels

Units such as **cup, tablespoon, teaspoon, tbsp, ** etc., may occasionally be misclassified as ingredients, especially when the token appears close to ingredient names or when the dataset contains variations in unit representation. This is partly due to limited unit examples in the dataset compared to ingredients.

4. Context Plays a Critical Role in Prediction

The contextual features used in the model, such as previous and next tokens, helped the CRF learn relationships between tokens in ingredient lists. For example, a quantity token is usually followed by a unit or ingredient, and a unit is often followed by an ingredient. However, when the surrounding context is unusual or inconsistent, prediction errors can occur.

5. Impact of Class Weights on Model Learning

Applying inverse frequency class weights helped reduce bias toward the dominant ingredient label. This improved the model's ability to recognise less frequent labels like quantity and unit, though some imbalance effects may still remain.

6. Common Error Patterns Observed

From the error dataframe analysis, the following patterns were noticed:

- * Some units are predicted as ingredients due to similar contextual placement.
- * Certain numeric tokens without clear unit context may be misclassified.
- * Tokens with ambiguous meaning depending on context can lead to prediction errors.

7. Opportunities for Model Improvement

The model performance could be improved by:

- * Expanding the unit keyword dictionary with more variations.
- * Incorporating additional contextual features such as bi-grams or phrase-level patterns.
- * Increasing the size and diversity of the training dataset.
- * Using domain-specific embeddings or transformer-based models in future iterations.

Overall, the CRF model demonstrates a good baseline performance for recipe entity recognition, successfully extracting key components such as ingredients, quantities, and measurement units from unstructured recipe text.

10. Conclusion

A Named Entity Recognition (NER) system was developed to automatically extract key entities such as ingredients, quantities, and measurement units from recipe ingredient lists. The model was implemented using Conditional Random Fields (CRF), which is well suited for sequence labeling tasks where the prediction of a token depends on its surrounding context.

The workflow began with data ingestion and preprocessing, where the JSON recipe dataset was loaded into a dataframe and tokenized into `input_tokens` and `pos_tokens`. Data validation steps ensured that the token lengths between input and labels were consistent, and invalid rows were removed to maintain data quality.

Next, the dataset was split into training and validation sets (70:30 ratio) to evaluate the model's ability to generalize to unseen data. Exploratory Data Analysis (EDA) was conducted to understand the distribution of ingredients, units, and quantities. This analysis revealed that ingredient labels dominate the dataset, highlighting the need to address class imbalance during model training.

To improve the model's predictive capability, a comprehensive feature engineering strategy was implemented. Features included lexical attributes (token, lemma, shape), grammatical features (POS tag, dependency relation), and contextual features (previous and next tokens). Additional domain-specific features were added to detect quantities and measurement units using keyword lists and

regular expression patterns. Class imbalance was handled by applying inverse frequency-based class weights, ensuring that less frequent labels such as unit and quantity received greater importance during training.

The CRF model was then trained using the lbfgs optimization algorithm with L1 and L2 regularization to prevent overfitting. Model performance was evaluated using flat classification reports and confusion matrices for both the training and validation datasets. The results showed that the model performed well in identifying ingredient entities, while the recognition of units and quantities improved due to the introduction of weighted features and contextual information.

Further analysis was conducted using error analysis on the validation dataset, where misclassified tokens were identified and examined along with their surrounding context. This analysis helped reveal common error patterns such as confusion between units and ingredients, or difficulties in identifying numeric expressions without clear contextual clues.

Overall, the developed CRF-based NER model successfully demonstrates the ability to convert unstructured recipe ingredient lists into structured entity information. Such structured data can be used in real-world applications such as recipe management systems, dietary tracking platforms, and food e-commerce applications.

For future improvements, the model could benefit from:

- * larger and more diverse training datasets
- * additional domain-specific features
- * incorporation of advanced deep learning models such as BiLSTM-CRF or transformer-based NER models

Despite these potential enhancements, the current implementation provides a strong baseline solution for extracting structured information from recipe text using traditional NLP techniques.